CS315 · ALGORITHMS ANALYSIS & DESIGN

Lecture 5 Greedy Algorithms

ملخص الميد · CLRS Chapter 16

محارب

محتوى الملف

01 الفكرة

02 عناصر الـ Greedy

03 Activity-selection

04 Knapsack

05 Huffman codes

01 الفكرة

الـ Greedy ياخذ أحسن خيار قدامه الحين، وما يرجع عن قراره أبداً، وما يهتم إذا الخيار هذا يوصل للحل الأمثل في النهاية.

A greedy algorithm selects the best option available at the moment. It never reverses an earlier decision, even if the choice is wrong. It works in a **top-down** approach.

التطبيقات: **Activity-selection** و **Knapsack** و **Huffman codes**.

02 عناصر الـ Greedy

الخاصية	معناها
Greedy-choice property	نوصل للحل الأمثل الكلي باختيار الأفضل محلياً، بدون ما نشوف نتائج المسائل الأصغر.
Optimal substructure	الحل الأمثل فيه حلول مثلى للمسائل الأصغر (مشتركة مع الـ DP).

	Greedy	Dynamic programming
القرار	خيار واحد الحين، بدون رجوع	يجرب كل الخيارات ويخزن النتائج
الاتجاه	Top-down	غالباً bottom-up
الحل الأمثل	مو دايماً	دايماً (لو الخاصيتين متوفرة)

Activity-selection 03

أنشطة كلها تحتاج نفس المكان (قاعة وحدة). كل نشاط له بداية `s` ونهاية `f` . نبي أكبر عدد أنشطة ما تتداخل.

Two activities are **compatible** if their intervals [si, fi) and [sj, fj) do not overlap.
Greedy choice: sort by finish time; pick the activity that finishes first, remove the ones that overlap it, repeat.

مثال السلايد

a	1	2	3	4	5	6	7	8	9	10	11
s	1	3	0	5	3	5	6	8	8	2	12
f	4	5	6	7	9	9	10	11	12	14	16

```
take a1 (finishes at 4)
a2, a3 start before 4 → rejected
a4 starts at 5 ≥ 4 → take a4 (finishes 7)
a5, a6, a7 start before 7 → rejected
a8 starts at 8 ≥ 7 → take a8 (finishes 11)
a9, a10 start before 11 → rejected
a11 starts at 12 ≥ 11 → take a11
```

```
Answer: {a1, a4, a8, a11}  →  4 activities
```

فيه حلول ثانية بنفس العدد مثل `{a2, a4, a9, a11}` .

```
GREEDY-ACTIVITY-SELECTOR(s, f)
1  n = s.length
2  A = {a1}
3  k = 1
4  for m = 2 to n
5      if s[m] ≥ f[k]
6          A = A ∪ {am}
7          k = m
8  return A
```

وقت التشغيل

النسخة العادية والـ recursive كلها `θ(n)` لو الأنشطة مرتبة حسب `f` ، لأن كل نشاط نشوفه مرة وحدة. لو مو مرتبة نضيف الترتيب `O(n log n)` .

"The recursive activity-selection algorithm takes O(n log n)" → **Other value**: Θ(n)

Knapsack 04

شنطة تشيل وزن أقصى `W` . كل غرض له قيمة `v` ووزن `w` . نبي أكبر قيمة.

	Fractional knapsack	knapsack 0-1
الأغراض	نقدر ناخذ جزء من الغرض	الغرض كامل أو لا (0 أو 1)
الحل	**Greedy** (يعطي الأمثل)	**DP** (الـ Greedy ممكن يغلط)
الوقت	`O(n lg n)` بسبب الترتيب	

طريقة الـ Greedy: احسب $d = \frac{v}{w}$ (القيمة لكل وحدة وزن)، رتّب من الأكبر، وخذ بالترتيب لين تمتلي.

مثال السلايد: W = 50

الغرض	w	v	$d = \frac{v}{w}$
1	10	60	6
2	20	100	5
3	30	120	4

0-1, greedy:	take 1 and 2 →	60 + 100 = 160 (20 lbs left empty)
0-1, optimal:	take 2 and 3 →	100 + 120 = 220
other option:	take 1 and 3 →	60 + 120 = 180
Fractional:	$1 + 2 + \frac{20}{30}$ of 3 →	60 + 100 + 80 = 240

يعني الـ Greedy فشل في 0-1 (المساحة الفاضية تنزل القيمة)، وهنا نحتاج DP لأن كل غرض نجرب ناخذه أو لا، والمسائل الصغيرة تتكرر.

جاء في الميد

462: Saffron و Cardamom و Cinnamon و Pepper في شنطة 5 كيلو → 5100. و 461 → 60 Fractional knapsack. محلولة في ملفات الحل.

05 Huffman codes

الهدف: نضغط النص. ASCII يعطي كل حرف 8 بت. Huffman يعطي الحرف المتكرر كود قصير، والنادر كود أطول.

الطريقة

1. اكتب تكرار كل حرف ورتبها تصاعدي.

2. خذ أصغر اثنين، اجمعهم في عقدة جديدة.

3. رجّع العقدة الجديدة للقائمة، وكرر لين تبقى عقدة وحدة (الجذر).

4. اليسار = 0 واليمين = 1، والكود = المسار من الجذر للحرف.

مثال السلايد: BCCABBDDAECCBBAEDDCC (20 حرف)

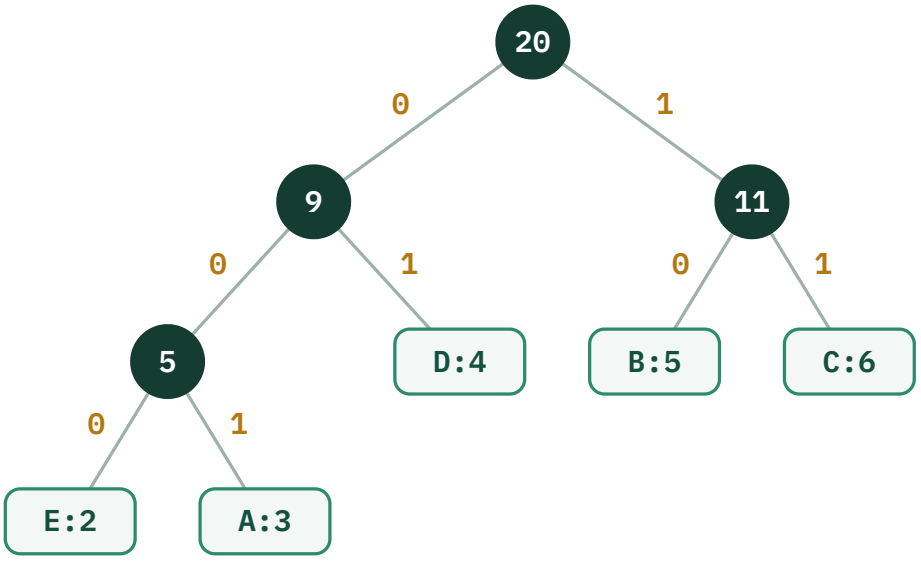
```
Counts:  E = 2,  A = 3,  D = 4,  B = 5,  C = 6
```

```
E + A = 5
```

```
5 + D = 9
```

```
B + C = 11
```

```
9 + 11 = 20  (root)
```



الحرف	التكرار	الكود	البتات
A	3	001	3 × 3 = 9
B	5	10	5 × 2 = 10
C	6	11	6 × 2 = 12
D	4	01	4 × 2 = 8
E	2	000	2 × 3 = 6

```
Huffman:  9 + 10 + 12 + 8 + 6 = 45 bits
```

```
ASCII:    20 letters × 8 = 160 bits
```

احفظ

Time complexity of Huffman coding: **O(n log n)**, n = number of unique characters.
The code length depends on the frequency of occurrence of characters. → **True**
No code is a prefix of another code (prefix code).

```
HUFFMAN(C)
1  n = |C|
2  Q = C                        // min-priority queue by frequency
3  for i = 1 to n − 1
4      z = new node
5      z.left  = x = EXTRACT-MIN(Q)
6      z.right = y = EXTRACT-MIN(Q)
7      z.freq = x.freq + y.freq
8      INSERT(Q, z)
9  return EXTRACT-MIN(Q)        // the root
```